iPRA with inferred elimination premises

Smaller proof checkers below the audited comparison envelope

Prepared for Vladimir Reshetnikov

12 November 2026

Abstract

We simplify a complete proof-certificate presentation of intuitionistic primitive recursive arithmetic by reconstructing an elimination's major formula from its target and substitution data. The major must still be proved. The resulting programs reduce the identified compact comparison envelope from 256 significant Python tokens, 278 AST nodes and 506 normalized bytes to 253 tokens, 275 nodes and 503 bytes, with different programs attaining the three coordinates. A separate complete native public-goal checker falls from 656 to 643 bytes, also improving both other coordinates. We give universal simulation arguments, executable certificate translations, frozen controls, and the full inherited logical and arithmetic bi-interpretation. Both numerical domains are whole, equality is actual, and both composites have graph $x = y$. The mathematical axioms are inherited; the new results concern certificate structure and trusted implementation size. No absolute minimum or new axiom deletion is claimed.

Comparison snapshot: `62396b06020f794fa6dc01b4025a6a5db9d4d28a`

Standalone LaTeX source, compiled PDF, proof checkers, controls and validation artifacts.

Contents

1 Result and its exact scope **5**
1.1 What has become smaller . . . 5
1.2 The statement established . . . 5

2 The complete native theory **6**
2.1 Nonlogical axiom schemas . . . 6
2.2 Logical and structural inference rules . . . 6

3 Why this is full intuitionistic arithmetic **7**
3.1 Equation induction and numerical cases . . . 7
3.2 All six ordinary logical operations . . . 7
3.3 The arithmetic bridge and both composites . . . 8

4 The compact public presentation **9**
4.1 Finite array representation . . . 9
4.2 The current seed-selected computation packets . . . 9

5 New result: infer the elimination major **10**

6 New result: a smaller byte implementation **11**

7 A smaller complete native public-goal checker **12**

8 Measurements and reproducibility **13**
8.1 Executable verification . . . 14

9 Assessment and remaining limits **14**

A Attribution and reading the inherited appendices **16**

B Complementary introductions with one local assumption **16**
B.1 Complete native syntax and rule inventory . . . 16
B.2 Exact equivalence and the unused-witness construction . . . 17
B.3 Induction and numerical cases separate cleanly . . . 18
B.4 The entire ordinary logical interface . . . 19
B.5 Whole-domain realizations and both formula composites . . . 20
B.6 What the theorem does and does not claim . . . 21

C Arithmetic strength and both ordinary interpretation composites **21**
C.1 The arithmetic dependency order . . . 22

D Internal scalar predecessor bootstrap **23**
D.1 Elementary terms and two finite cycles . . . 23
D.2 A ramp with a global scaling identity . . . 24
D.3 A comparator from scalar iteration . . . 25
D.4 Saturated iteration gives predecessor . . . 26
D.5 Dependency boundary . . . 27

E From literal-equation induction to full intuitionistic PRA **27**
E.1 The exact hypotheses . . . 27
E.2 Constructor separation and two elementary tests . . . 28
E.3 Four elementary uses of equation induction . . . 29
E.4 An equation invariant that reflects actual equality . . . 30
E.5 Compilation of all quantifier-free formulas . . . 31
E.6 Recovery of induction and the full theory . . . 32

F Pairing, primitive recursion, and internal numerical composites **33**
F.1 Pairing from a bounded square-root search . . . 33
F.2 Compiling each primitive recursion . . . 34
F.3 Why both numerical round trips are internal . . . 34

G A guarded core with a deterministic introduction rule **34**
G.1 Complete syntax, axioms, and inference rules . . . 34
G.2 Padding, equation implication, and numerical cases . . . 36
G.3 Recovery of contextual equation induction . . . 37
G.4 Exact translations to and from the inherited calculus . . . 37
G.5 Why an unrestricted checker can still be sound . . . 39
G.6 Why merely deleting the choice would not suffice . . . 40

H Removing literal zero from the source of transport **40**
H.1 Structural operations, cases, and equation induction . . . 41
H.2 The equality operations available immediately . . . 41
H.3 A singleton-one test without predecessor . . . 42
H.4 Turning a zero-source equality around . . . 43
H.5 Recovering equality algebra and formula transport . . . 44
H.6 Compatibility with restricted computation certificates . . . 45

H.7 Scope of the result 45

I A deterministic calculus with auxiliary formulas 45

I.1 Terms, public formulas, and proof formulas 46
I.2 All primitive axioms and rules 46
I.3 Induction and cases are available before equality is repaired 47
I.4 Recovering equality from the restricted source rule 47
I.5 Soundness of every auxiliary derivation 49
I.6 Completeness and both interpretation composites 50
I.7 Why the auxiliary distinction is essential 51

J A uniform arity encoding and its checking theorem 51

J.1 The complete sorted interpretation of arrays 52
J.2 One depth calculation replaces the lookup table 53
J.3 Opening and scope 53
J.4 A computation hypothesis only needs a zero instance 54
J.5 Audit of the logical continuations 55
J.6 Canonical completeness, including zero witnesses 56
J.7 JSON and finite-runtime qualifications 57

K Complete new checker listings 59

K.1 Inferred-major token profile 59
K.2 Inferred-major AST profile 59
K.3 Truthy-result byte profile 60
K.4 Byte profile, ordinary decrement 60
K.5 Native public-goal profile 60

1 Result and its exact scope

This report gives a smaller implementation of a complete presentation of intuitionistic primitive recursive arithmetic. The mathematical theory is the native scalar-iteration calculus already developed in ProveIt. The new contribution is a simplification of its proof-certificate protocol, together with two independently justified implementation refinements. No arithmetic axiom deletion and no theorem of absolute minimality is claimed.

The comparison is pinned to commit `62396b06020f794fa6dc01b4025a6a5db9d4d28a`, whose ConciseIPRA directory contains ninety research packages. Smaller implementation supplements inside those packages matter: the README table alone does not give the attained comparison frontier.[1] In particular, the byte target is 506, not 521 or 516; the complete native public-goal checker target is 656. The relevant source files, their hashes, and the measurements are included in this archive.

1.1 What has become smaller

An elimination certificate formerly supplied both a template with its endpoints and a redundant copy of the major formula. Once the proposed target instance matches the current goal, that major can be reconstructed. The verifier still demands a proof of the reconstructed formula. Removing the comparison against the redundant copy reduces the token and abstract syntax tree coordinates. This is a deterministic change in the certificate protocol, rather than an invitation to search for an omitted proof.

A separate implementation represents successful recursive checks by nonempty result objects and sequences the assumption construction and its continuation inside an ordered pair. This reduces the normalized byte coordinate without changing the certificate language. Finally, a small Boolean-order identity reduces the complete native public-goal validator while preserving its entire finite-JSON acceptance relation.

> **Comparison boundary.** The compact profiles have the inherited guarded public-language contract and do not separately validate that public grammar. The native profile performs the inherited full Eq/D public-goal grammar and scope validation. These two boundaries are measured separately throughout.

The three compact coordinate improvements are attained by different programs. There is no claim that one program simultaneously has the smallest token, node, and byte counts. Each proposed profile nevertheless implements the same public theoremhood relation, through its specified certificate protocol. The native profile gives a separate, stronger input boundary and improves all three of its incumbent's coordinates in one program. The full measurements appear in Section 8.

1.2 The statement established

Let $\mathcal{N}$ be the native calculus specified below, and let $\mathcal{A}$ be its root-selected auxiliary calculus restricted to guarded public conclusions, as specified in Appendix I. For the appropriate public encoding of a closed formula A:

(i) the new native checker has an accepting finite certificate exactly when the old native checker does;

(ii) the inferred-major compact checkers have an accepting certificate exactly when $\mathcal{A}$ proves A;

(iii) the byte-profile transformation preserves acceptance of every finite certificate for its control, under the same ideal resource convention;

(iv) the ordinary realizations of these public presentations are bi-interpretable and equiconsistent with full first-order iPRA.

Both numerical interpretation domains in (iv) are the whole original domain, equality is actual equality, and both numerical isomorphisms have the graph $x = y$. The last statement is inherited through exact proof translations; this report supplies the new checker equivalences and reproduces the logical and arithmetic bridge in its appendices. No new Lean or Rocq formalization is asserted.

2 The complete native theory

There is one numerical sort. Terms and formulas are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

In an iterator only u in h is bound; its counter and initial value are ordinary terms. There are no function-valued object variables and no universal evaluator. A particular iterator is a finite syntactic description with fixed numerical parameters. Put $1 = S0$.

The intended ordinary readings are

$$\mathsf{Q}_y B(y) \;\equiv\; B(0) \to \forall y B(y), \qquad \mathsf{D}_{x,y} B(x,y) \;\equiv\; \exists x (B(x,0) \to \forall y B(x,y)).$$

The designated contradiction in native syntax is $0 = 1$. Contexts Γ are finite lists of reusable formulas. All substitutions are capture-free; bound variables are renamed before substitution.

2.1 Nonlogical axiom schemas

Exactly two computation schemas are primitive:

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0,IS}$$

They are available in every retained context. In particular, successor injectivity, zero separation, pairing, predecessor, and an equality test are not additional primitive numerical axioms of this native presentation.

2.2 Logical and structural inference rules

Besides computation, the primitive patterns are the following.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \text{Hyp}, \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B \quad \Gamma, \mathsf{Q}_y B(b,y) \vdash A}{\Gamma \vdash A}\ \text{DE},$$

$$\frac{\Gamma, B(t,0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}\ \text{DI}_{\text{case}},$$

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}E(x,y)} \quad \mathrm{DI}_{\mathrm{progress}}.$$

Here F is any native formula template, while E in the last rule must be a literal equation. In DE the parameter b is fresh for the retained context, the conclusion, and the fixed parameters of B. In either DI rule, z is fresh for the context, witness t, and fixed parameters of the matrix. The displayed local assumptions are discharged. The two DI patterns form one introduction family with a checked mode bit.

There are no other primitive logical axioms or inference rules. Weakening, exchange, contraction, renaming, cut, and numerical substitution are admissible for this unrestricted native presentation by induction on finite derivations. They are metatheorems, not uncharged executable instructions. In presentations with a syntactic source restriction, substitution must instead be recovered by a separate admissibility argument; the appendices give that argument where it is needed.

The auxiliary Q can be eliminated from public formulas: $\mathsf{Q}_y B(y)$ is interderivable with $\mathsf{D}_{v,y}B(y)$ for an unused v. The checkers therefore use Eq/D as their native public language while retaining Q as an internal resource.

3 Why this is full intuitionistic arithmetic

It is necessary to recover all first-order intuitionistic connectives, not just an equational or negative fragment. Here are the constructions and the dependency order; the detailed inherited proofs are reproduced after the new results.

3.1 Equation induction and numerical cases

For a literal equation $E(y)$, introduce $\mathsf{D}_{x,y}E(y)$ by progress with unused x. Eliminate it to obtain the closure $\mathsf{Q}_y E(y)$. Supplying $E(0)$ and applying QE yields $E(t)$. Thus contextual equation induction is derived. Cases for an arbitrary formula are obtained in the same way using case DI, ignoring its local zero assumption when proving the successor case. Freshness is retained in both arguments.

Reflexivity follows from computation and transport. With $r = I(0,t;u.0)$, use $r = t$ as both transport premises and the template $v = t$ to obtain $t = t$. Standard templates then give symmetry, transitivity, and congruence. Transport under a binder is part of the native rule; reduced variants recover it explicitly.

3.2 All six ordinary logical operations

Let $C(v,a,b) = I(v,a;u.b)$, with unused u. Its two computations select a at zero and b at a successor. Define a gate by

$$G_v(s = t) = (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}G_v B.$$

Then $G_0 A$ is derivable and $G_{Sz}A$ is interderivable with A. One also constructs, by external recursion on the finite formula syntax, a selector $M_v(A,B)$ with endpoints $M_0(A,B) \dashv\vdash A$ and $M_{Sz}(A,B) \dashv\vdash B$. For equations it applies C to both sides; when binder depths differ, the shorter formula is padded by a vacuous gated binder. The complete recursion is given in Appendix B.4.

Let $p(z) = I(z, 0; u.C(u, 1, 0))$. The formulas

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &= (0 = 1) \end{aligned}$$

have all the ordinary intuitionistic introduction and elimination rules. Every newly introduced binder is fresh, and u is unused. For conjunction, the instances $p(1) = 1$ and $p(2) = 0$ recover its two components. For the universal quantifier, the first conjunct supplies zero and the second supplies every successor; numerical coverage then supplies every element. No excluded middle for quantified formulas is used.

3.3 The arithmetic bridge and both composites

Define the ordinary theory T_I using intuitionistic first-order equality logic, the same staged iterator equations, contextual equation induction, and the two sentences

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is essential: a merely positive equational theory would have a singleton model. In native syntax its translation is the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$, because native falsity is that equation. This explicitly reconciles ordinary falsity and native contradiction.

The internal arithmetic proof first constructs a scalar predecessor, then a characteristic of actual equality, then recovers quantifier-free induction, and finally constructs pairing and compiles primitive recursion. This is the dependency order of the proof. In particular, predecessor is not constructed using an assumed faithful equality detector or an assumed pairing function. Each fixed primitive-recursive description is compiled to a scalar iterator using derived pairing to store its counter and current result. All passive parameters remain fixed during induction.

In the reverse interpretation, iPRA defines every fixed scalar iterator by primitive recursion that ignores the recursion-counter argument in its update. Equation induction proves uniqueness of each fixed iterator and of each primitive recursion. External induction on descriptions therefore shows, internally in both theories, that a twice-translated operation equals the original operation at every argument. The numerical domain is unchanged, so the composite isomorphism is exactly $x = y$.

For formulas the essential composite is

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big).$$

One direction opens the native closure and introduces the three macros; the other eliminates the existential and introduces D using the implication and universal premise. External induction on formulas and the endpoint properties of G, M give both formula composites. The explicit falsity bridge then yields effective contradiction-preserving proof translations and equiconsistency. These arguments do not assert that either theory proves its own consistency.

4 The compact public presentation

The compact profiles use the same numerical language and a guarded public formula language $\mathcal{G}$. Every equation is public. A formula $\mathsf{D}_{x,y}B$ is public if B is compound and public, or if it is literally

$$I(y, 0; u.h) = I(y, 0; u.k).$$

The update bodies may contain their state variable and all displayed parameters. Its zero instance is provable by computation. Auxiliary proofs may additionally use unrestricted Eq/Q/D formulas.

In the auxiliary calculus, transport has a source other than the literal 0, and one introduction chooses progress for a literal equation and cases for a compound matrix. This is a complete presentation on *public* sequents. It is not asserted to be equivalent on every auxiliary raw formula. The public guard is mathematically relevant, not an optional output formatting convention.

For clarity, the translation of an arbitrary native formula uses the padding

$$L(E) = \mathsf{D}_{u,v}G_vE, \qquad P(A) = \begin{cases} L(A), & A \text{ an equation,} \\ A, & A \text{ compound.} \end{cases}$$

Padding is interderivable with the original formula. First replace every auxiliary Q_yB by $\mathsf{D}_{v,y}B$ with unused fresh v. On the resulting Eq/D syntax define $T(E) = E$ for equations and $T(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}P(T(B))$. By induction $T(B)$ is public; its padding is compound and public, so the outer binder is also public. The guarded arithmetic basis recovers equation induction, numerical cases, and both directions of padding. The inherited source-restriction argument restores zero-source transport without adding an axiom. The appendices prove the translation and its formula round trips.

4.1 Finite array representation

The certificate is a JSON array. Its last element is the goal; preceding four-field packets are consumed from the end. An unused prefix is allowed by the inherited contract. Success is zero process exit status together with exactly `ACCEPT` on standard output.

In the uniform arity codec a unary array represents a paired binder, a binary array an equation, and a ternary array an internal closure:

$$[B] \leftrightarrow \mathsf{D}_{x,y}B, \qquad [s, t] \leftrightarrow s = t, \qquad [B, B, B] \leftrightarrow \mathsf{Q}_yB.$$

In numerical position, a unary array is successor and a ternary array is the initialized scalar iterator. Array nodes adjust the opening depth by $5 - \mathrm{len}(a)$; paired numerical slots have width two. Negative integer labels are allocated eigenparameters. The scope floor decreases before the continuation opens its fresh assumption.

The representation contains administrative truth and zero aliases. Their complete sorted reading, the dummy slots in successor and in the iterator's counter and seed, and the known-zero-instance supply lemma are reproduced in Appendix J. No lemma equating arbitrary raw spellings is assumed. Public meanings and proof-level administrative objects remain distinct in the correctness statement.

4.2 The current seed-selected computation packets

The inherited arity appendix records an older packet arrangement. The measured controls use the following equivalent seed-selected arrangement, also documented verbatim in `reference/packet_FORMAT.md`.

A packet is $[m, s, u, F]$. After any transfer, a nonzero major is proved and its triplication is retained: a unary paired formula supplies its ternary closure, while the other triplications are administrative truth entries. If the major is falsy and the initializer field u is truthy, the generated computation equation is

$$\begin{cases} I(0, u; v.F) = u, & s \text{ falsy}, \\ I(Ss, u; v.F) = F[I(s, u; v.F)/v], & s \text{ truthy}. \end{cases}$$

If both m and u are falsy, the current unary goal is introduced with witness F. The native profile's falsy field distinguishes the two permitted introduction modes; the compact profile reads the mode from the matrix root. The common continuation then allocates a fresh parameter and checks the successor goal under the selected opened assumption.

This arithmetic dispatch does not remove the computation equations whose initializer or successor predecessor is literally zero. Put $W = I(2, 1; v.0)$. Its successor computation is an allowed direct packet and proves $W = 0$. Given an omitted computation instance, replace just its forbidden zero initializer and/or predecessor by the same placeholder w, leaving passive zeros in its update unchanged. The resulting equation $E(W)$ is an allowed direct computation. Transport along $W = 0$ proves $E(0)$, the required instance. Thus two computation leaves and one transport recover every omitted instance, without deciding any term's numerical value.

Generated assumptions can contain the newly admitted parameter and need not be uniformly true in it. The precise justification is the known-zero-instance supply lemma: if b is fresh for Γ, A, $H(0)$ is proved, and $\Gamma, H(b) \vdash A$, substitute zero for b in the continuation and cut in $H(0)$. The sorted opener proof establishes that a successful raw computation supply has such a zero instance. This is why changing the order of scope allocation and opening would require a new proof. The transformations below preserve that order.

5 New result: infer the elimination major

Write $i(F, t)$ for the existing capture-safe opening of the raw template F at the numerical term t, at the current scope floor. A packet has fields (m, s, u, F). The old transfer test was

$$m = M(s, u, F) \quad \text{and} \quad i(F, u) = A,$$

where

$$M(s, u, F) = \begin{cases} [s, u], & s \text{ is truthy in the codec}, \\ [F, F, F], & s \text{ is falsy in the codec}. \end{cases}$$

The discriminator is certificate truthiness (in particular, an empty array is falsy), not a decision about the arithmetic value of a term. On canonical payloads it distinguishes the literal zero from other term constructors. The first alternative requires equality transport and the second closure specialization. In either case the verifier checks the major and then continues with $i(F, s)$.

The new protocol makes the target match decisive:

$$\boxed{i(F, u) = A \quad \Longrightarrow \quad m := M(s, u, F), \quad A := i(F, s).}$$

The reconstructed m is subsequently proved by exactly the same recursive major-premise checker. A supplied packet field cannot assert the reconstructed premise without proving it. When the target does not match, the packet retains the old interpretation.

Theorem 5.1 (Soundness by effective-major expansion)**.** *Every accepting inferred-major certificate over the documented finite integer/list language expands to an accepting certificate for the matched old checker, proving the same public goal.*

Proof. Follow the finite successful execution. At each packet at which the target matches, replace its stored major by $M(s, u, F)$; at other packets leave it unchanged. Copy the successful major subexecution and the continuation recursively. The old checker now passes both comparisons, constructs the same assumption, and performs the same openings. If the target fails, its old conjunction fails and neither protocol transfers. The scope floor, context, and packet-consumption order therefore agree by induction on the finite execution tree. The transformation is implemented outside the trusted checker in `translate_inferred.py`. □

Lemma 5.2 (Separation of numerical payloads and meaningful goals)**.** *On canonical numerical payloads, $i(F, u)$ evaluated at formula depth zero cannot equal a meaningful Eq/*Q*/*D *goal.*

Proof. A numerical tree has only unary and ternary array nodes. Starting at even formula depth, their depth increments are respectively four and two, so depth remains even. At leaves, the opening either preserves a protected integer or supplies the formula-neutral zero; it does not insert the replacement image at a meaningful numerical hole. Hence the result still has no binary node. Every meaningful native formula has a binary equation node at the bottom of its finite chain of binders. Structural equality between the two trees is therefore impossible. □

Theorem 5.3 (Completeness of inferred majors)**.** *Every theorem of the guarded public calculus has a finite inferred-major certificate. Thus the new protocol presents the same public theory as its matched compact control.*

Proof. Use the inherited canonical proof compiler. In a transport or closure-use packet the target matches and the inferred major is exactly its required major. In a computation or introduction packet the fourth field is a numerical payload, so the preceding lemma prevents an accidental match. An ordinary paired elimination chooses the inert payload $F = 0$; its goal is meaningful and cannot equal that scalar. Hypothesis closure consumes no packet. These cases cover the compiled derivation. In the local profile neutral majors close using the retained scalar zero; the global profile skips proof recursion for a falsy major. The existence of the old canonical compiler and the guarded interpretation then establish completeness, rather than an empirical assertion about the regression suite. □

This theorem is deliberately not a claim of identical acceptance on every old raw certificate. The protocol now accepts some packets with a redundant major field different from the inferred major. The expansion proof establishes soundness, and the canonical compiler establishes completeness. The acceptance change is exhibited by a concrete certificate in the validation artifacts.

6 New result: a smaller byte implementation

The incumbent recursive checker returns `None` when successful. Its major-supply expression therefore uses `c(m,g,k) or m*3`: after verification the expression has the value $m * 3$. The new byte profile makes every successful return truthy: it returns `True` on a hypothesis, and otherwise a singleton list containing an ordered pair. The pair's first component computes the assumption supply; its second performs the recursive continuation.

Consequently the matching expression is `c(m,g,k) and m*3`. If $m = 0$, it still evaluates to zero and the old arithmetic/introduction alternatives are selected. If $m \neq 0$, it evaluates to the same tripled major. A failed recursive verification still raises; failure is never represented by a false return value that could be mistaken for an axiom branch.

Theorem 6.1 (Exact successful-execution correspondence)**.** *The byte-profile checker and its 506-byte control accept the same finite certificates, idealizing sufficient memory and recursion resources. They consume the same suffix of the certificate and check the same scope and context conditions in the same order.*

Proof. Induct on the finite successful invocation tree in either direction. A hypothesis changes the return value from `None` to `True`, without reading a packet. Otherwise the singleton comprehension reads one packet. Its ordered pair first computes the supply expression. Successful major checks are truthy by induction, so the Boolean expression produces exactly the old supply value. A DI assignment to the goal occurs at this point, before the continuation.

Python evaluates the pair's second component only after its first. Within the continuation call its arguments are evaluated left to right. Thus source/template opening still sees the old floor, then the fresh floor is assigned, and the final goal and new assumption openings see that new floor. The local opener's closure observes the same floor updates as in the control. The recursive invocation therefore receives the same goal, context, and floor; induction applies.

Every successful nonhypothesis return is a list of exactly one pair, hence truthy independently of the pair's contents. Conversely a completed pair contains a completed recursive continuation, so the argument reverses. No exception is caught by this transformation, and the driver emits `ACCEPT` only after the root call completes. This proves the claimed acceptance equivalence and consumption correspondence. □

The 503-byte profile retains the incumbent spelling `~-k` for $k - 1$. The scope floor is always an integer, so these expressions agree. A companion uses the ordinary spelling $k-1$ and measures 505 bytes; it also beats the old 506-byte profile. The 505-byte companion is preferable when source readability matters more than the two additional bytes. Neither profile improves every coordinate of the incumbent; the token and AST improvements come from the inferred-major profiles.

7 A smaller complete native public-goal checker

The separate native checker validates the Eq/D goal before interpreting its proof. Its inherited opener carries a private mode v: initially $v = 1$, and $v = 0$ throughout subsequent proof checking. The first scan checks list arities, canonical integer-coordinate spellings, and scope. The existing depth parity records the formula/term sort, so no independent parser or grammar file is imported.

For a Boolean-valued predicate P, the guard

$$\neg v \vee P \qquad \text{is equivalent to} \qquad v \leq P \quad (v \in \{0, 1\}).$$

The new implementation uses that ordered Boolean guard in the two validation positions and replaces a join of two bits by bitwise union. The required safety fact is not merely Boolean algebra: the right-hand predicate is now evaluated even when $v = 0$.

At a list branch, the left discriminator has already established an actual list, so the arity predicate is total. At a scalar branch, the scope comparisons have succeeded; the remaining public predicate first

tests exact integer type and then numerical parity and bounds, so it is total whenever reached. The predicates are side-effect free. At the fallback, both operands are bits ($d \bmod 2$ and v), so logical and bitwise union have the same zero/nonzero result.

Theorem 7.1 (Native public-checker preservation)**.** *The new native profile accepts exactly the finite JSON certificates accepted by the pinned 656-byte native control, under the same resource idealization. In particular it retains the complete public-goal grammar and scope validation, and it presents the same full native theory.*

Proof. The private mode invariant follows directly from the driver, default argument, and reset before the first packet. The preceding totality and Boolean identities prove equality of the initial validation outcome and all subsequent opening outcomes. Every packet read, major recursion, assumption supply, scope change, and continuation is otherwise unchanged. Induction on a finite consumed execution establishes the equivalence. □

"Complete public-goal validation" has its precise inherited meaning. Unused certificate prefixes are allowed, and not every irrelevant internal JSON value is lexically revalidated. The new implementation does not relax those conditions; its entire input acceptance relation equals its control's. Nor does this establish a unique spelling for all dummy numerical slots. The abstract native language and the sorted certificate interpretation specify their meaning.

8 Measurements and reproducibility

All figures count an entire standalone acceptance module, including its JSON input and success verdict. Each uses only the Python standard library. The shared Python runtime and standard library are excluded on both sides, exactly as in the pinned comparison corpus. No proof producer, translator, normalizer, reference file, or test helper is imported by an acceptance module.

The metric script removes actual docstrings, applies `ast.unparse`, adds one LF, and counts UTF-8 bytes, significant Python tokens, and AST nodes. Comments, whitespace tokens, and end markers do not count as significant tokens. The reference runtime is CPython 3.12.14. The archive contains the measured sources, metrics in machine-readable form, and the measurement script. Source is compiled before measurement is accepted; parsing into an AST alone does not establish that Python accepts the program.

Complete module	Tokens	AST nodes	Bytes
Token control	256	283	594
Inferred-major token profile	253	280	599
AST control	260	278	602
Inferred-major AST profile	257	275	607
Byte control	267	303	506
Truthy-result byte profile	278	310	503
Byte profile, ordinary decrement	278	309	505
Native public-goal control	333	379	656
Native public-goal profile	327	375	643

The identified compact coordinate envelope moves from

$$(256, 278, 506) \quad \text{to} \quad (253, 275, 503)$$

in the order significant tokens, AST nodes, normalized bytes. The ordinary decrement companion gives a 505-byte option. These are coordinatewise attained bounds, obtained by different source programs. A single joint minimum is not claimed. Likewise the audit establishes the listed incumbents, not an exhaustive lower-bound theorem over all possible implementations or every historical file in the repository.

The native profile improves its matching fully validated incumbent from $(333, 379, 656)$ to $(327, 375, 643)$. Its larger size is not evidence that it implements a weaker mathematical theory: it performs additional input validation and accepts unrestricted native Eq/D public goals. It should not be compared directly to a compact profile that assumes the guarded public contract.

8.1 Executable verification

The delivered harness compiles each standalone source, remeasures it, runs inherited positive and negative certificates, tests the inferred-major expansion, and compares the exact-preservation refinements against their controls. The suite contains real proof steps using computation, introduction, elimination, equality, and scope allocation, as well as malformed or unproved goals. A specific changed-major example demonstrates that the new inference protocol is exercised.

The validation results are recorded in `evidence/validation.json`. They supplement the universal proofs above. A finite test suite cannot establish soundness, completeness, bi-interpretability, or a global source minimum. Ordinary and optimized Python runs are included where relevant to check that acceptance does not depend on assertions being enabled.

To reproduce the deliverable, run

```
python3 reproduce.py
python3 reproduce.py --pdf
```

The first command performs the executable checks and measurements; the second additionally compiles this source with pdfLaTeX. The provided PDF is compiled from the provided LaTeX, with its pages rendered and inspected. The source includes its appendices and checker listings directly and requires no network access to compile.

9 Assessment and remaining limits

Under the user's stipulated implementation measure, the report makes a strict reduction below each identified compact coordinate incumbent, and a reduction of all three coordinates of one complete native public-goal checker. The changes are small and exactly described. The inferred-major protocol removes redundant certificate data; the other two refinements preserve existing certificates. These are the new results.

The work does not establish an absolutely simplest axiomatization, a global program-size lower bound, a single program simultaneously below all three compact incumbents, or a new deletion of a primitive native axiom. It does not extend the guarded public bi-interpretation to every administrative raw object. It also does not treat the inherited Lean development as a machine verification of the new Python source.

The ordinary theory is fully specified, the two interpretations and their composites are included, and the new implementation arguments preserve that mathematical content. That is a reproducible

improvement under the specified metric; its limited mathematical novelty should remain visible alongside the numerical improvement.

A Attribution and reading the inherited appendices

The following nine sections reproduce the pinned ProveIt foundations, with their original mathematical prose and internal cross-references. Their statements described as "new" or "the new calculus" refer to the historical packages from which they came, not to the present report. They are included so that the interpretation and arithmetic dependencies can be checked without retrieving another archive. The MIT No Attribution license is supplied as `LICENSE-ProveIt.txt`; exact source paths and Git blob hashes are in `evidence/source_provenance.json`.

The contribution of the present report is confined to Sections 5, 6, and 7, their measurements, and their executable artifacts. The predecessor, equality characteristic, logical macros, guarded translation, source admissibility, and scalar primitive-recursion compiler are inherited. This distinction is also recorded in `CLAIMS.json`.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two

equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [8] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0+b=b$ and $Sa+b=S(a+b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a+a$ and $3a$ means $a+a+a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n,0;u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n,0;u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n,0;u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0)=1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n)+2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N(I(Sy, 2E(x); H)) \\ &= N(I(y, H(2E(x)); H)) \\ &= N(I(y, 3E(x); H)) \\ &= N(3I(y, E(x); H)) \\ &= N(I(y, E(x); H)) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [10]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE

contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$
$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G A guarded core with a deterministic introduction rule

The change is to the public formula language and the introduction rule, not to the numerical domain. An atomic matrix immediately below a paired binder must have a fixed, syntactically recognizable zero guard. Such a matrix has a provable zero instance. Consequently it needs only the progress introduction; the separate atomic case introduction can be deleted. Compound matrices retain the case introduction.

G.1 Complete syntax, axioms, and inference rules

Numerical terms are those of the inherited scalar calculus:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the displayed variable u is bound in h. There is one numerical sort, fixed finite term syntax, and capture-free substitution. Put

$$C(v,a,b) := I(v,a;u.b), \qquad 1 := S0,$$

where u is fresh. For a literal equation $E \equiv (s = t)$, define

$$G_v E \;:=\; \big(C(v,0,s) = C(v,0,t)\big). \tag{Guard}$$

The displayed abbreviations expand into terms of the given syntax.

Call a literal equation a *zero-initialized matrix in y* when it has the syntactic form

$$B(x,y) \equiv I(y,0;u.h) = I(y,0;u.k). \tag{Canonical atomic matrix}$$

Here h, k may depend arbitrarily on the state variable u, on x, y, and on fixed parameters; state binders are renamed as necessary. The selector guard $G_y E$ is a special case, with state-independent updates. Every matrix of (Canonical atomic matrix) has the theorem $B(x,0)$ by the two zero computations and equality.

The canonical formulas of $\mathcal{G}$ are generated as follows:

1. Every literal equation $s = t$ is a formula.
2. $\mathsf{D}_{x,y}B$ is a formula when B is already a compound canonical formula (its root is D).
3. $\mathsf{D}_{x,y}B(x,y)$ is a formula whenever B has the zero-initialized shape (Canonical atomic matrix).

The bound variables x, y are distinct. Rule 3 is the restriction on atomic matrices. It does not require an equality decision procedure or a proof that an arbitrary base is true: the guard has the displayed literal shape. The designated contradiction is $0 = 1$.

Retained contexts contain reusable clauses $\langle A; y.B\rangle$, where A, B are canonical formulas and $y \notin \mathrm{FV}(A)$. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where v is fresh and H is the fixed computation theorem $I(0,0;u.0) = 0$.

The two nonlogical axiom schemas, available in every context, are

$$I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u]. \tag{I0,IS}$$

Besides these, there are precisely the following rules:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } \langle A; y.B(y)\rangle \in \Gamma \qquad \text{(Use)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

and the single, root-determined introduction schema

$$\frac{\Gamma, B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{$\mathrm{DI}_{\mathrm{root}}$}$$

There is no certificate choice between these two cases. At an atomic matrix the guard condition on the conclusion is mandatory in the canonical calculus. In DE, a is fresh for Γ, F and the fixed

parameters. In DI, z is fresh for Γ, t and the fixed parameters. The displayed local resources are discharged. Transport ranges over canonical formula templates, and capture-free numerical substitution preserves their guard shape.

There are no additional primitive logical rules, reflexivity axioms, successor axioms, coverage axioms, or induction schemas. Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on derivations. For cut, replace a use of the corresponding constant clause by the supplied proof, renaming eigenparameters first.

G.2 Padding, equation implication, and numerical cases

Write $A \dashv\vdash B$ for interderivability under arbitrary additional retained context. The following preliminary arguments use no contextual induction rule.

Lemma G.1 (Equality and guard endpoints)**.** *Reflexivity, symmetry, transitivity, numerical congruence, and their usual contextual uses are derivable. Moreover,*

$$\vdash G_0 E, \qquad G_{Sz} E \dashv\vdash E.$$

Proof. For reflexivity, put $r = I(0, t; u.0)$. The computation equation $r = t$ serves as both premises of transport with motive $v = t$, yielding $t = t$. Standard equality motives then give the remaining equality rules. The two guard claims follow from the two computation equations for C, together with equality transport. □

For an equation E let

$$L(E) := \mathsf{D}_{u,v} G_v E, \qquad P(A) := \begin{cases} L(A), & A \text{ is an equation,} \\ A, & A \text{ is compound.} \end{cases} \tag{Padding}$$

The newly bound u, v are fresh for E. In particular, $P(A)$ is always compound.

Lemma G.2 (Padding)**.** *$L(E) \dashv\vdash E$, and therefore $P(A) \dashv\vdash A$.*

Proof. Under E, atomic progress introduction derives $L(E)$: its target $G_{Sz}E$ follows from the retained E, independently of the local induction assumption. Conversely, DE on $L(E)$ opens $\langle G_0 E; v.G_v E\rangle$. Supply the guard-zero theorem and use the clause at 1; guard removal gives E. □

We will need to reason by numerical cases under an assumption that contains the case variable. It is essential not to violate the eigenvariable condition by directly retaining that assumption during case introduction. A small implication macro resolves this issue.

For equations $U \equiv (s = t)$ and $V \equiv (r = q)$ put

$$M_w(U, V) := (C(w, s, r) = C(w, t, q)), \qquad U \Rightarrow_g V := \mathsf{D}_{a,w} L(M_w(U, V)), \tag{Equation implication}$$

with fresh padding variables and unused a. The matrix of the outer binder is compound.

Lemma G.3 (Implication between equations)**.** *The macros in* (Equation implication) *admit both rules*

$$\frac{\Gamma, U \vdash V}{\Gamma \vdash U \Rightarrow_g V}, \qquad \frac{\Gamma \vdash U \Rightarrow_g V \qquad \Gamma \vdash U}{\Gamma \vdash V}.$$

Proof. The endpoint identities for C and padding give $L(M_0(U,V)) \dashv\vdash U$ and $L(M_{Sz}(U,V)) \dashv\vdash V$. Compound case DI therefore gives introduction. DE, the first endpoint, Use at 1, and the second endpoint give elimination. These endpoint conversions use numerical transport with parameters outside the fresh padding binders; they do not assume a binder-congruence theorem. □

Lemma G.4 (Numerical cases)**.** *For every canonical $A(y)$ the following rule is admissible:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{NC}$$

Proof. First suppose A is compound. Case DI with an unused witness coordinate introduces $\mathsf{D}_{u,y}A(y)$, ignoring its local base assumption in the successor premise. DE opens $\langle A(0); y.A(y)\rangle$; supply the given base proof and use it at t.

If A is an equation, apply this already proved compound case rule to $L(A(y))$. Padding converts the two premises to its endpoints and converts the conclusion back. No progress induction is being used in this construction beyond the special padding introductions. □

G.3 Recovery of contextual equation induction

Theorem G.5 (Equation induction)**.** *For every literal equation $E(y)$, $\mathcal{G}$ derives*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{EI}$$

Proof. Put $J(y) = G_y E(y)$. We first prove the progress premise $\Gamma, J(z) \vdash J(Sz)$ without making an invalid contextual use of NC. Consider the compound formula

$$\Phi(y) := J(y) \Rightarrow_g E(Sy).$$

Under Γ, prove $\Phi(0)$ by equation-implication introduction: ignore its local hypothesis, use the supplied proof of $E(0)$, and apply the supplied step proof specialized at 0 to obtain $E(1)$.

For fresh w, prove $\Phi(Sw)$ by equation-implication introduction. Its local hypothesis $J(Sw)$ gives $E(Sw)$ by guard removal, and the supplied step proof specialized at Sw gives $E(SSw)$. These two branch proofs have retained context Γ, independent of the case variable. Thus NC legitimately gives $\Phi(z)$ under Γ. Under the local hypothesis $J(z)$, equation-implication elimination gives $E(Sz)$, hence $J(Sz)$ by the successor guard endpoint.

Atomic progress DI now introduces $\mathsf{D}_{u,y}J(y)$ with unused u. DE and the theorem $J(0)$ yield $J(Sw)$, and hence $E(Sw)$, for a fresh w. Finally NC for E uses the supplied base $E(0)$ and these successor instances to conclude $E(t)$.

The only induction-like premise used here is the single displayed guarded progress introduction. In particular, no predecessor, coverage disjunction, formula induction, or pre-existing EI is assumed. □

G.4 Exact translations to and from the inherited calculus

Let $\mathcal{C}$ be the inherited Choice-DI calculus: the same scalar terms, arbitrary formulas $s = t$ and $\mathsf{D}_{x,y}B$, the same computation, Use, DE, and transport rules, with case DI at every matrix and progress DI at literal equations. Define $T : \mathcal{C} \to \mathcal{G}$ on finite formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}P(TB). \tag{Translation}$$

Each padding binder uses fresh names. This is a computable structural translation, preserves free numerical variables, commutes with capture-free numerical substitution up to bound-variable renaming, and translates every paired matrix to a compound matrix. Translate clauses by $T\langle A; y.B\rangle = \langle TA; y.TB\rangle$. Let $J : \mathcal{G} \to \mathcal{C}$ be literal inclusion.

Theorem G.6 (Translation of derivations)**.**

$$\Gamma \vdash_{\mathcal{C}} A \Longrightarrow T\Gamma \vdash_{\mathcal{G}} TA, \qquad \Gamma \vdash_{\mathcal{G}} A \Longrightarrow J\Gamma \vdash_{\mathcal{C}} JA.$$

The transformations are effective on finite derivations.

Proof. Inclusion is immediate: every guarded rule is an allowed Choice-DI rule. For T, induct on a Choice-DI derivation.

Computation is unchanged. Use commutes with substitution, and transport uses the translated motive. A case DI premise translates to a proof from $TB(t, 0)$ to $TB(t, Sz)$. Under the local assumption $P(TB)(t, 0)$, unpad, apply that translated proof, and pad the result. Compound case DI then proves the required $\mathsf{D}_{x,y}P(TB)$.

For a progress DI with equation matrix E, its translated step is still $\Gamma, E(t, z) \vdash E(t, Sz)$. To introduce $\mathsf{D}_{x,y}L(E(x, y))$, use compound case DI. Its retained base $L(E(t, 0))$ supplies $E(t, 0)$. The recovered contextual EI, under that base, gives $E(t, Sz)$, which padding converts to the target $L(E(t, Sz))$.

For DE, the translated major is $\mathsf{D}_{x,y}P(TB)$. It opens the clause $\langle P(TB)(a, 0); y.P(TB)(a, y)\rangle$. The recursively translated minor expects $\langle TB(a, 0); y.TB(a, y)\rangle$. Replace every use of this expected clause by: pad its antecedent, use the opened clause at the same term, and unpad its consequent. This effective local proof replacement preserves the fresh witness condition and the conclusion. □

Lemma G.7 (Compound-matrix congruence)**.** *If canonical compound matrices B, C are uniformly interderivable, then $\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}C$ in $\mathcal{G}$.*

Proof. Open the source binder at a fresh witness a. Introduce the target by compound case DI at that witness. Convert its retained base to the source base, use the opened source clause at Sz, and convert the successor instance to the target instance. Close DE. The reverse conversion proves the converse. □

Lemma G.8 (The guarded atomic composite)**.** *For an admissible zero-initialized atomic matrix $B(x, y)$,*

$$\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}L(B) \quad \textit{in } \mathcal{G}. \qquad \text{(Atomic composite)}$$

Proof. Forwards, open the source clause at a. Introduce the target by compound case DI: unpad its base, use the source clause at Sz, and pad the result.

Backwards, open the clause of $\mathsf{D}_{x,y}L(B)$ at a. The two zero computations prove $B(a, 0)$, and padding proves $L(B(a, 0))$. Thus the opened clause gives $L(B(a, Sz))$, and unpadding gives $B(a, Sz)$, independently of the local assumption $B(a, z)$. Atomic progress DI therefore introduces $\mathsf{D}_{x,y}B$ at witness a. Close DE. This argument uses the true base explicitly; it would fail for an arbitrary unguarded atomic matrix. □

Theorem G.9 (Both composites and bi-interpretability)**.** *For all $\mathcal{C}$-formulas A and all canonical $\mathcal{G}$-formulas B,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{G}} TJ(B).$$

Consequently $\mathcal{C}$ and $\mathcal{G}$ admit effective structural intertranslations preserving their full consequence relations, with both native formula composites and literal identity numerical translations.

Proof. In $\mathcal{C}$, padding is interderivable with its argument, and case introduction proves binder congruence for arbitrary matrices. Structural induction gives the first displayed equivalence.

For the second, induct on a canonical B. Equations are unchanged. At a compound matrix, the induction hypothesis and compound-matrix congruence apply. At a guarded atomic matrix, the statement is exactly (Atomic composite). The equivalences are uniform under retained assumptions, so they extend to clauses and sequent contexts by the same local replacement of Use instances.

Both interpretations leave the numerical sort, every term, and numerical equality untouched. Both numerical composites therefore have the identity graph $x = y$; no quotient, retract, numeral restriction, or extra coding sort is involved. □

For the first-order bi-interpretability statement, use the ordinary realization that reads every canonical $\mathsf{D}_{x,y}B$ uniformly as $\exists x(B(x,0) \to \forall y B(x,y))$ and leaves numerical equality and all scalar terms unchanged. Composing the displayed translations and composites with the inherited full-domain bridge between $\mathcal{C}$ and iPRA yields a bi-interpretation of this ordinary realization of $\mathcal{G}$ with full first-order iPRA. The contradiction $0 = 1$ is unchanged, so equiconsistency follows as well. The inherited arithmetic bridge supplies the primitive-recursive compiler and the ordinary logical interpretation; this section does not claim those inherited constructions as new.

G.5 Why an unrestricted checker can still be sound

There is a useful, but strictly separate, administrative calculus $\mathcal{U}$. It permits arbitrary atomic paired matrices and uses the same root-determined DI, with ordinary Q-style clauses on DE. It is not claimed that all of $\mathcal{U}$ is bi-interpretable with iPRA.

Give arbitrary administrative formulas their following ordinary interpretations:

$$(s = t)^\dagger = (s = t),$$

$$(\mathsf{D}_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is a literal equation,} \\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound.} \end{cases} \quad \text{(Mixed administrative reading)}$$

Retained clauses keep the ordinary reading $A^\dagger \to \forall y\, B^\dagger$. If the raw codec also permits administrative Q-closures as formula-shaped objects, including under transport or a paired binder, extend this interpretation by $(Q_y B)^\dagger = B^\dagger(0) \to \forall y B^\dagger(y)$. Such a matrix is compound for root-determined DI. This adds no canonical object-formula constructor and preserves the following soundness argument.

Every $\mathcal{U}$ inference is sound in iPRA under this reading. Atomic DI directly proves its progressive reading; compound case DI uses numerical coverage. In atomic DE, contextual equation induction turns $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, exactly the clause that DE supplies. All other rules are immediate, including transport: numerical substitution does not change whether a matrix is an equation.

On canonical zero-initialized matrices, the mixed and ordinary readings agree provably in iPRA. Indeed $B(x,0)$ is a theorem, so, with x fixed,

$$\forall y\, (B(x,y) \to B(x,Sy)) \quad \longleftrightarrow \quad B(x,0) \to \forall y\, B(x,y).$$

The forward direction is equation induction; the reverse uses the true base to obtain every instance. Structural induction extends this agreement to all canonical guarded formulas.

Thus administrative proofs with unguarded intermediate formulas are conservative on canonical guarded conclusions: translate an accepted derivation into iPRA, use the canonical agreement, and translate

back through the established bi-interpretation. Conversely, every canonical guarded derivation is already an administrative derivation.

This justifies a checker contract consisting of soundness and completeness for canonical guarded statements while allowing broader administrative certificates. It does *not* make the raw checker a validator of the guarded public grammar. It also does not assert completeness for every noncanonical encoding under the mixed reading. If a comparison requires rejection of all noncanonical public goals, the corresponding syntax validator must be implemented and counted.

G.6 Why merely deleting the choice would not suffice

Without the guarded canonical language, root-deterministic DI does not preserve the original native consequence relation. Let

$$p(y) = I(y, 0; u.C(u, 1, 0)), \qquad E(y) \equiv (p(y) = 1),$$

and leave the existential witness coordinate unused. Computation gives $p(0) = 0, p(1) = 1, p(2) = 0$. The original case introduction proves $\mathsf{D}_{x,y}E(y)$: its base gives $0 = 1$, from which equality transport and selector computation prove every equation.

By contrast, the progressive reading of this formula is false in the standard natural numbers, because $E(1)$ holds and $E(2)$ does not. The mixed interpretation proves that it is not a theorem of the unrestricted root-deterministic calculus. The guard, the explicit translations, and both composite proofs are therefore substantive parts of the reduction, rather than optional notation.

H Removing literal zero from the source of transport

The following is a restriction of a mathematical inference rule, rather than a change in the spelling of zero. Throughout this section, "nonzero source" means *not syntactically the single term* 0. Every variable and every term headed by S or I is a permitted source, including terms whose numerical value is zero.

Let $\mathcal{C}$ be the native scalar calculus with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

the two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

formulas generated by equality and the Q, D binders, and the Hyp, QE, DE, case-DI and equation-progress-DI rules stated in the main inventory. The same argument applies to the original presentation with D object formulas and Q resources in contexts. Let $\mathcal{C}^-$ replace unrestricted transport by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr}^-, \qquad s \not\equiv 0. \tag{Tr$^-$}$$

Here F is an arbitrary native numerical template, and $s \not\equiv 0$ is a condition on finite term syntax. All other rules are unchanged.

Theorem H.1 (Literal-zero source elimination)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{C}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A.$$

In particular, the ordinary whole-domain realization of $\mathcal{C}^-$ *is bi-interpretable and equiconsistent with full first-order iPRA through the same interpretations as the native scalar core. No change of numerical domain or interpreted equality is needed.*

We prove the theorem without assuming arbitrary numerical substitution admissibility for $\mathcal{C}^-$: such substitution can turn a variable source into the forbidden literal zero. Every occurrence of a helper equation at an arbitrary term is either obtained directly from its displayed derived rule at that term, or from a derivation whose transport sources retain an S or I head under the indicated substitution.

H.1 Structural operations, cases, and equation induction

Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. These operations do not turn a nonzero source into literal zero. As usual, fresh parameters are renamed before a cut. We also use the following restricted substitution fact: a capture-free substitution is admissible for a particular finite derivation if it leaves every primitive transport source in that derivation syntactically different from zero. This fact follows by the same induction; it is not unrestricted substitution admissibility.

Lemma H.2 (Cases and equation induction need no transport)**.** *In* $\mathcal{C}^-$ *the following rules are derivable:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}\ \mathsf{NC}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ \mathsf{EI}.$$

In NC, A *is any native formula; in EI,* E *is one literal equation. The parameter* z *is fresh for* Γ*,* t*, and the fixed parameters of the matrix. Other parameters stay fixed throughout EI.*

Proof. Choose an unused witness coordinate x. In NC, weaken the successor proof by $A(0)$ and apply case-DI, with witness zero, to obtain $D_{x,y}A(y)$. DE opens $Q_yA(y)$, because x is unused. QE with the given base proof and target t yields $A(t)$, and DE closes. For EI use progress-DI instead, with the given equation step, and use the same DE/QE construction. No transport rule occurs. □

These rules will be applied to unconditional helper equations before they are weakened into the context containing $0 = t$. We never use NC with a case parameter occurring in its retained context.

H.2 The equality operations available immediately

Lemma H.3 (Reflexivity and safe equality operations)**.** *Reflexivity* $r = r$ *is derivable for every term* r*. From* $p = q$ *with* $p \not\equiv 0$*, one can derive* $q = p$ *and* $H(p) = H(q)$ *for any numerical template* H*. Every direct computation can be used in either direction as an equation; using the reversed equation as a transport major is permitted only when its new source is syntactically nonzero.*

Proof. Put $b = I(0, r; u.0)$. The computation $b = r$, used as both major and minor of transport with template $v = r$, gives $r = r$. Its source is the nonempty iterator term b. For symmetry of $p = q$, transport the reflexivity $p = p$ through the template $v = p$. For numerical congruence, transport the reflexivity $H(p) = H(p)$ through $H(p) = H(v)$. Both use the permitted source p. Every computation equation has an I-headed left side, so its symmetry follows by the same construction. □

An occurrence-selective template may keep some occurrences of a term fixed while abstracting other occurrences. This is ordinary capture-free template substitution, not an extra inference rule.

Lemma H.4 (Joining two nonzero terms known to be zero)**.** *Suppose $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$. Then $\Gamma \vdash p = q$ using only Tr^- and computation.*

Proof. Let

$$z_0 = I(0, 0; u.0), \qquad b_p = I(0, p; u.0).$$

Computation gives $b_p = p$. Start with $b_p = b_p$ and use the major $p = 0$ in the template $I(0, v; u.0) = b_p$. This gives $z_0 = b_p$. Symmetry at the nonzero source z_0 gives $b_p = z_0$. Now transport through $v = z_0$ using the computation $b_p = p$, whose source is b_p, to obtain $p = z_0$. The identical construction gives $q = z_0$. Symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ gives $p = q$. The sources used are p, q, z_0, b_p, b_q; all are syntactically nonzero. No reverse transport from zero has been used. □

The zero-join lemma is deliberately placed before the next construction. The positive case of its singleton test needs precisely this lemma; ordinary transitivity through zero is not yet available.

H.3 A singleton-one test without predecessor

Define the finite scalar terms

$$\begin{aligned} C(v, a, b) &:= I(v, a; u.b), & Z(v) &:= C(v, 1, 0), \\ K(v) &:= C(v, 0, 1), & H(v) &:= C(v, 1, 2), \\ N(n) &:= I(n, 0; u.Z(u)), & R(n) &:= I(n, 0; u.H(u)), \\ L(n) &:= N(R(n)). \end{aligned} \tag{Tests}$$

The update variable in C is unused. Here $1 = S0$ and $2 = S1$. The standard-model description of R is $0, 1, 2, 2, \ldots$; the proof below establishes the needed equations internally and does not assume this description.

Lemma H.5 (Uniform test equations)**.** *The following equations have derivations in $\mathcal{C}^-$, uniformly in the displayed argument:*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$

$$N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad \boxed{L(Sv) = Z(v)}. \tag{Singleton}$$

All transport sources in these helper derivations have an I or S head, except for the explicit nonzero endpoints in the zero-join lemma; those endpoints also have an I head in this application.

Proof. For $H(v) = SK(v)$ use NC. At zero both sides reduce to 1; at a successor both reduce to 2. To join two equations $p = j$, $q = j$ with $j = 1$ or $j = 2$, reverse $q = j$ at its nonzero source q, then transport at the nonzero numeral j. Each reduction of these particular terms is a forward replacement of an iterator computation. This proves the helper at an arbitrary target v by NC, without substituting zero into a variable-source transport.

The equations $N(1) = 1$ and $N(2) = 0$ are finite computations: $N(0) = 0$, $N(1) = Z(N(0)) = 1$, and $N(2) = Z(N(1)) = 0$. Every replacement source in these calculations is an iterator term. Likewise $R(1) = H(R(0)) = H(0) = 1$.

Computation and the first helper give, for any v,

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Use this equation at the nonzero source $R(Sv)$ in $R(SSv) = H(R(Sv))$. The resulting right side is $H(SK(R(v))) = 2$ by the successor computation of C. Thus $R(SSv) = 2$. These substitutions are all at I-headed sources. Substitute $R(1) = 1$ and $R(SSv) = 2$ inside N and use the two computed values of N to obtain $L(1) = 1$ and $L(SSv) = 0$.

Finally apply NC to the matrix $L(Sv) = Z(v)$. At zero, $L(1) = 1$ and $Z(0) = 1$ are joined through the nonzero numeral 1. At a successor, $L(SSv) = 0$ and $Z(Sv) = 0$ are joined by Lemma H.4; both left sides are nonempty iterator terms. This proves the boxed equation at every target. Its use is therefore an instance of a displayed derived rule, not an appeal to unrestricted substitution admissibility. □

Lemma H.6 (An unconditional zero-filter identity)**.** *For every term t,*

$$t = C(Z(t), t, 0) \tag{Filter}$$

is derivable in the empty context of $\mathcal{C}^-$.

Proof. Apply NC to $v = C(Z(v), v, 0)$, with the target t. At zero, forward computations give $C(Z(0), 0, 0) = C(1, 0, 0) = 0$; reverse this equation at its nonzero left side. At Sv, forward computations give $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$; again reverse at the nonzero iterator left side. Thus the base and successor proofs have no retained assumption depending on the case variable. All numerical arguments of the final NC target are permitted. In particular, this identity is proved before any assumption $0 = t$ is used. □

H.4 Turning a zero-source equality around

Lemma H.7 (Zero-source symmetry)**.** *For every term t,*

$$0 = t \ \vdash_{\mathcal{C}^-} \ t = 0. \tag{ZS}$$

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Work in the fixed context $\Gamma = \{0 = t\}$ and put

$$A(n) := I(n, t; u.Su).$$

We first prove $n = A(n)$ by EI, keeping t fixed.

For the base, computation gives $A(0) = t$. Its symmetry gives $t = A(0)$. Since $t \not\equiv 0$, transport at source t in the retained assumption $0 = t$ gives $0 = A(0)$.

For the step, retain $z = A(z)$ with z fresh. This source is a variable, hence is permitted. Numerical successor congruence from Lemma H.3 gives $Sz = SA(z)$. Computation gives $A(Sz) = SA(z)$, and its symmetry gives $SA(z) = A(Sz)$. Its source $SA(z)$ is also permitted, so transport in $Sz = SA(z)$ gives $Sz = A(Sz)$. This is exactly the EI step. The induction hypothesis is used at the same fixed parameter t.

At target 1, EI gives $1 = A(1)$. Forward computation gives $A(1) = St$: the successor equation produces $SA(0)$, and the nonzero-source computation $A(0) = t$ is used inside S. Transport at source $A(1)$ consequently gives

$$1 = St.$$

Its source 1 is permitted. Numerical congruence for L gives $L(1) = L(St)$. Use $L(1) = 1$ at source $L(1)$, and $L(St) = Z(t)$ at source $L(St)$, obtaining $1 = Z(t)$. Symmetry at source 1 gives $Z(t) = 1$.

Weaken the unconditional Filter identity into Γ. Transport at the iterator source $Z(t)$ changes its right side to $C(1, t, 0)$. The latter computes to zero, and its source is again an iterator term. The resulting equation is $t = 0$, as required. □

This proof uses cases only for unconditional helper equations. The sole induction with the retained assumption $0 = t$ has the literal equation $n = A(n)$ as invariant. It does not perform a forbidden case distinction while silently changing the retained context.

H.5 Recovering equality algebra and formula transport

Lemma H.8 (Full numerical equality algebra)**.** *The calculus $\mathcal{C}^-$ admits unrestricted symmetry, numerical congruence, and transitivity. In particular it admits root replacement*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}.$$

Proof. Symmetry uses Lemma H.3 when the source is not zero and Lemma H.7 otherwise.

For numerical congruence from $s = t$, a nonzero s is handled by Lemma H.3. If $s \equiv t \equiv 0$, use reflexivity. If $s \equiv 0$ and $t \not\equiv 0$, first derive $t = 0$, apply permitted numerical congruence at source t to obtain $H(t) = H(0)$, and use the now established full symmetry.

For transitivity from $r = s$ and $s = t$, a nonzero middle term s is handled by one permitted transport. If $s \equiv 0$ and either endpoint is literally zero, a given premise is already the conclusion. Otherwise $r, t \not\equiv 0$; reverse $0 = t$ to get $t = 0$ and apply the zero-join lemma to $r = 0$ and $t = 0$. Root replacement follows by symmetry and transitivity. These constructions are acyclic: zero-source symmetry was proved before any unrestricted numerical congruence or transitivity. □

Lemma H.9 (Full transport)**.** *For every native numerical template $F(v)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in $\mathcal{C}^-$, without a source restriction.

Proof. Use external induction on the finite number of formula binders in F, uniformly in retained contexts and fixed numerical parameters. For an equation $F(v) \equiv A(v) = B(v)$, use unrestricted numerical congruence to obtain $A(s) = A(t)$ and $B(s) = B(t)$. Join these with $A(s) = B(s)$ using the established equality algebra.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh a, retaining $Q_yB(s, a, y)$. Introduce the desired $F(t)$ by case-DI with witness a. Its local assumption is $B(t, a, 0)$. Full symmetry gives $t = s$, and the external induction hypothesis for the smaller matrix transports the assumption to $B(s, a, 0)$. QE gives $B(s, a, Sz)$, and the induction hypothesis in the forward direction gives $B(t, a, Sz)$. Close case-DI and DE. All new parameters can be chosen fresh for the source equality and the retained context.

If Q is first-class formula syntax, first observe that its case introduction is derived without transport: introduce $D_{u,y}B(y)$ with unused u by case-DI and eliminate it with the one-node continuation $Q_yB(y)$. For $F(v) = Q_yB(v, y)$, use this derived introduction for the target. Convert its base assumption backwards by the induction hypothesis, apply the old Q resource, and convert the successor conclusion forwards. This is the same smaller-matrix argument and uses no new numerical induction. □

Proof of Theorem H.1. Every restricted derivation is an unrestricted one. Conversely, replace each unrestricted transport node by the derivation supplied by Lemma H.9, recursively translating its premises first. Other inference nodes are unchanged. The result has the same context and conclusion. Thus the full native consequence relation, not just closed theorems, is preserved. The existing native/ordinary and ordinary/iPRA interpretations and both identity composites therefore transfer unchanged. □

H.6 Compatibility with restricted computation certificates

Some compact checkers require the initializer of a computation packet to have nonempty syntax, and encode a successor counter by a nonempty predecessor. These are certificate restrictions, not restrictions on the two mathematical computation schemas. They do not introduce a circular dependence on zero-source transport in the preceding proof.

Use the inherited certified-zero term

$$c_0 = I(2, 1; u.0).$$

Its equation $c_0 = 0$ is a directly admitted successor computation: the predecessor and initializer are both 1. For an arbitrary computation instance with literal-zero initializer or literal-zero successor predecessor, form its whole equation as a template $J(v)$, replacing just these offending occurrences by v. The instance $J(c_0)$ is a directly admitted computation. Transport through this equation template using $c_0 = 0$ yields $J(0)$, the desired original instance. Both computation premises are admitted directly, and the only transport source is the nonempty term c_0.

This is exactly the inherited three-node normalization in `Research/certified_zero/tests/producer.py`, function `expand_computation`: one transport and two computation premises. Its source is c_0, even when both defects are corrected simultaneously. Passive zero occurrences in an update are left alone. Thus all computation instances used in the mathematical proof can be expanded before compiling to the source-restricted checker.

H.7 Scope of the result

The new statement is the admissibility of all omitted literal-zero source transports. The native term/formula language, arithmetic schemas, and target theory remain the inherited ones. This theorem justifies replacing a two-way equality/Q major-pattern test by a source-discriminated test in a suitable certificate protocol. It does not assert a minimum proof-checker size, a shortest proof expansion, or a new characterization of the primitive recursive functions. The displayed proof is a written universal derivation; executable certificates, when supplied, validate particular instances rather than substitute for this argument.

I A deterministic calculus with auxiliary formulas

The construction combines two restrictions that have different roles. The source of an equality elimination must not be the literal term 0, and the root of a paired matrix determines its introduction rule. There is no independent case/progress choice. The interaction is resolved by allowing auxiliary formulas in proofs. Only the public formulas below are assigned the ordinary conditional reading in the bi-interpretation theorem. In particular, we do not assert a round-trip theorem for every auxiliary formula.

I.1 Terms, public formulas, and proof formulas

There is one numerical sort. The terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where u is bound only in h. Substitution is capture-free. Write

$$1 := S0, \qquad 2 := S1, \qquad C(v, a, b) := I(v, a; u.b),$$

with u fresh in the last abbreviation. For an equation $E \equiv (s = t)$, put

$$G_v E := \big(C(v, 0, s) = C(v, 0, t)\big).$$

The public language $\mathcal{G}$ is the guarded language of the inherited package: every equation is public; $\mathsf{D}_{x,y}B$ is public if B is a compound public formula; and an atomic matrix immediately under $\mathsf{D}_{x,y}$ must have the literal shape

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{G}$$

The bodies h, k may contain x, y, the state variable, and fixed parameters. This is a syntactic condition. Every such matrix has a provable zero instance. The designated contradiction is $0 = 1$.

The auxiliary proof language $\mathcal{U}$ is larger:

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

Its paired matrices need not satisfy (G). Contexts are finite lists of reusable auxiliary assumptions. Public theoremhood means that a public conclusion has a derivation in this auxiliary calculus; it does not require every intermediate formula to be public. The same definition applies to consequence from public assumptions.

I.2 All primitive axioms and rules

The only nonlogical axiom schemas, available under every context, are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0,IS}$$

The hypothesis rule is $\Gamma \vdash A$ whenever $A \in \Gamma$. The remaining rules are

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad (s \text{ is not literally } 0), \tag{Tr$^-$}$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}, \tag{QE}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(a, y) \vdash A}{\Gamma \vdash A}, \tag{DE}$$

and one root-determined introduction schema

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ has root } \mathsf{Q} \text{ or } \mathsf{D}. \end{cases} \tag{DI$_{\text{root}}$}$$

Here F is an auxiliary formula template. In DE, a is fresh for the retained context, the conclusion, and the fixed parameters. In DI, z is fresh for the retained context, the witness term t, and the fixed

parameters. The displayed local assumption is discharged. Bound names are always chosen to avoid capture.

There is no primitive equality reflexivity or successor axiom, no separate induction schema, no unrestricted atomic case introduction, and no primitive introduction rule for Q. Weakening, renaming, contraction, and replacement of a retained assumption by its proof are admissible by the usual induction on a finite derivation. No unrestricted numerical-substitution theorem for auxiliary derivations is needed below: a substitution could turn an allowed source variable into the literal 0, so a direct constructor-wise substitution argument would be invalid.

I.3 Induction and cases are available before equality is repaired

Lemma I.1 (Auxiliary equation induction)**.** *For every literal equation $E(y)$, the rule*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}$$

is admissible, with the usual fresh z. The retained context may contain arbitrary auxiliary formulas.

Proof. Apply atomic DI to $\mathsf{D}_{x,y}E(y)$, with x unused and witness 0. Its premise is precisely the supplied step. DE exports $\mathsf{Q}_y E(y)$; the fresh witness is absent from this formula. QE and the supplied base give $E(t)$. This derivation uses no transport at all. Its paired intermediate formula is generally auxiliary rather than public. □

Lemma I.2 (Numerical cases)**.** *From $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$, with z fresh, one may infer $\Gamma \vdash A(t)$ for every auxiliary $A(y)$.*

Proof. For an equation, use Lemma I.1 and ignore its local induction assumption. For a compound formula, case DI with an unused witness gives $\mathsf{D}_{x,y}A(y)$; DE and QE use the given base to obtain $A(t)$. □

This is the decisive bootstrap. We do not first try to pad the hypothesis $0 = t$ into a public compound formula. Auxiliary equation induction is already available, and supplies the atomic cases needed by the literal-zero-source argument.

These are the compatibility steps needed to combine the two previously separate reductions. The guarded public grammar is inherited from [5]; the source restriction is inherited from [6].

I.4 Recovering equality from the restricted source rule

The following proof reconstructs the zero-source argument in the pinned `source_dispatch` package and `NativeZeroSource.lean`. Its only non-equational ingredients are equation induction and numerical cases, now supplied by Lemmas I.1 and I.2. The formulas in the calculations below are equations. The new proof reuses this numerical argument only; the final unrestricted auxiliary-formula transport argument of the source package is not used [6, 7].

Lemma I.3 (Safe equality calculations)**.** *Reflexivity is derivable for every term. Symmetry is derivable for $p = q$ when p is not literally 0, and transitivity is derivable through any middle term that is not literally 0. Congruence and replacement on either side of an equation are available along an equation with such a source. If p, q are not literally 0, then*

$$\Gamma \vdash p = 0, \quad \Gamma \vdash q = 0 \quad \Longrightarrow \quad \Gamma \vdash p = q. \tag{Zero join}$$

Proof. Put $r = I(0, t; u.0)$. The axiom $r = t$, used as both premises of Tr^- with template $v = t$, gives $t = t$. The usual equality templates then give symmetry at an allowed source, transitivity through an allowed middle term, and congruence at an allowed source.

For the last assertion, let $W(v) = I(0, v; u.0)$ and $w = W(0)$. From $p = 0$ and $W(p) = W(p)$, replace p only on the left to obtain $w = W(p)$. Symmetry at w, followed by the computation $W(p) = p$ on the left, gives $p = w$. Similarly $q = w$. Symmetry at q and transitivity through w give $p = q$. All these sources and the middle term w are syntactically nonzero; no numerical inequality is asserted. □

Lemma I.4 (Zero-source symmetry)**.** *For every term t and every auxiliary context,*

$$\Gamma \vdash 0 = t \quad \Longrightarrow \quad \Gamma \vdash t = 0.$$

Proof. The case where t is literally 0 is reflexivity. Otherwise t is an allowed source. Write

$$A(n) = I(n, t; u.Su).$$

The zero computation and safe symmetry give $t = A(0)$, which may replace the right side of the assumed $0 = t$; thus $0 = A(0)$. Under $z = A(z)$, congruence at the variable source z gives $Sz = S(A(z))$, and the successor computation gives $Sz = A(Sz)$. Auxiliary equation induction therefore gives $1 = A(1)$ and hence

$$1 = St. \tag{47}$$

For completeness, the small test used to remove this successor is

$$\begin{aligned} Z(v) &= C(v, 1, 0), & K(v) &= C(v, 0, 1), & H(v) &= C(v, 1, 2), \\ N(n) &= I(n, 0; u.Z(u)), & R(n) &= I(n, 0; u.H(u)), & L(n) &= N(R(n)). \end{aligned}$$

Numerical cases and computation give $H(v) = S(K(v))$. Consequently $R(1) = 1$ and $R(SSv) = 2$. Computation gives $N(1) = 1$ and $N(2) = 0$, so $L(1) = 1$ and $L(SSv) = 0$. A second numerical case argument yields

$$L(Sv) = Z(v). \tag{48}$$

At the zero branch both sides reduce to 1; at the successor branch both sides reduce to 0, and (Zero join) joins them. All reductions use iterator sources. Similarly numerical cases prove the reversed filter identity

$$C(Z(v), v, 0) = v. \tag{49}$$

Its zero branch reduces via $Z(0) = 1$, and its successor branch via $Z(Sw) = 0$. Again every major used for reduction has a nonzero syntactic source.

Applying L to (47), using $L(1) = 1$ and (48), and using safe symmetry and transitivity, gives $Z(t) = 1$. Hence the left side $q = C(Z(t), t, 0)$ of (49) is also provably 0. The filter gives $q = t$. Symmetry at the iterator source q, followed by transitivity through q, yields $t = 0$.

Every identity just used is a schema constructed at the displayed terms. The argument does not assume that arbitrary restricted derivations may be numerically substituted. □

Corollary I.5 (Unrestricted equational transport)**.** *Symmetry, transitivity, term congruence, and transport in every equation template are admissible without a source restriction.*

Proof. Symmetry follows from Lemmas I.3 and I.4. For transitivity through literal 0, the cases of a literal-zero endpoint are immediate; otherwise reverse the second equation and use zero join. For congruence along $0 = t$ with t not literally 0, reverse to $t = 0$, use safe congruence, then use unrestricted symmetry. If $F(v)$ is $a(v) = b(v)$, term congruence, symmetry, and transitivity transform $F(s)$ to $F(t)$. □

Lemma I.6 (Transport in public formulas)**.** *The unrestricted transport rule is admissible whenever its template F is public. Thus numerical equality has its full ordinary use throughout the public language.*

Proof. Induct on the public formula template. The equation case is the preceding corollary. Suppose first that $F(v) = \mathsf{D}_{x,y}B(v,x,y)$ has compound matrix. Open $F(s)$ at a fresh witness a, obtaining the clause $\mathsf{Q}_y B(s,a,y)$. Introduce $F(t)$ at witness a by compound DI. Its base assumption $B(t,a,0)$ transfers, using the induction hypothesis and $t = s$, to $B(s,a,0)$. QE gives $B(s,a,Sz)$; the induction hypothesis and $s = t$ transfer this to $B(t,a,Sz)$.

If B is atomic, its public guard makes $B(s,a,0)$ a theorem. The opened source clause therefore gives $B(s,a,Sz)$ outright. Unrestricted equational transport gives $B(t,a,Sz)$. Atomic DI introduces the target, ignoring its local induction assumption. Close DE in both cases. Eigenvariables are chosen fresh for all fixed parameters. □

This lemma is intentionally about public templates. We do not need, and do not assert, unrestricted zero-source transport in every auxiliary formula containing an unguarded atomic paired matrix.

I.5 Soundness of every auxiliary derivation

Give auxiliary formulas the following ordinary first-order readings over the scalar terms:

$$
\begin{aligned}
(s = t)^\dagger &= (s = t),\\
(\mathsf{Q}_y B)^\dagger &= B^\dagger(0) \to \forall y\, B^\dagger(y),\\
(\mathsf{D}_{x,y}B)^\dagger &= \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is an equation},\\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound}. \end{cases}
\end{aligned}
$$

Numeric substitution preserves the root category of a formula, so this reading commutes with capture-free numeric substitution. The reading of Q is always conditional, including at an equation.

Theorem I.7 (Auxiliary soundness)**.** *If $\Gamma \vdash_{\mathcal{U}} A$, then iPRA proves $A^\dagger$ from $\Gamma^\dagger$, after the inherited primitive-recursive realization of scalar terms.*

Proof. Induct on the derivation. Hypothesis, computation, transport, and QE have their ordinary sound interpretations. Atomic DI directly proves the displayed progressive reading. Compound DI uses numerical coverage to extend its zero-to-successor premise to a zero-to-all conclusion. For DE at an atomic matrix, ordinary equation induction converts $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, which is exactly the exported Q assumption. This uses only quantifier-free induction, with the existential witness and other parameters fixed. DE at a compound matrix is ordinary existential elimination. All side assumptions and eigenvariable conditions are preserved. □

Let A° denote the *uniform ordinary reading* on public formulas: equality is actual numerical equality and every public $\mathsf{D}_{x,y}B$ means $\exists x(B^\circ(x,0) \to \forall y B^\circ(x,y))$.

Lemma I.8 (Agreement on public formulas)**.** *For every public A, iPRA proves $A^\dagger \leftrightarrow A^\circ$.*

Proof. At a guarded atomic matrix $E(x,y)$, the zero instance $E(x,0)$ is a theorem. Progress implies all instances by equation induction, while conditional closure implies all instances by that true base and hence implies progress. These two readings are therefore equivalent, with x fixed, and remain equivalent under the existential quantifier. At compound matrices use structural induction. Equations are unchanged. □

I.6 Completeness and both interpretation composites

We record the inherited guarded translation explicitly, because it identifies the exact scope of the result. Let $\mathcal{C}$ be the existing native scalar calculus. Define

$$L(E) = \mathsf{D}_{u,v} G_v E, \qquad P(A) = \begin{cases} L(A), & A \text{ an equation}, \\ A, & A \text{ compound}. \end{cases}$$

The new variables in L are fresh. Translate old Eq/D formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} P(TB).$$

If the old presentation also treats Q as an object constructor, put $T(\mathsf{Q}_y B) = \mathsf{D}_{x,y} P(TB)$ with x unused. Let J be the literal inclusion of public formulas into the old language.

Lemma I.9 (Guarded translation and its composites)**.** *The translation T preserves derivations of the old calculus in the new public consequence relation. Uniformly under additional contexts,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{U}} TJ(B) \quad (B \textit{ public}).$$

Proof. After equality repair, $L(E) \dashv\vdash E$: from E, atomic DI proves $L(E)$ by ignoring its local assumption; conversely DE, the theorem $G_0 E$, and QE at 1 recover E. Therefore $P(A)$ is always compound and interderivable with A.

Old case introduction transfers to compound DI after padding the base and the successor conclusion. Old equation progress transfers by auxiliary equation induction under the padded base assumption, followed by compound DI. The old direct Q progress rule is covered by Lemma I.1 and the unused-witness translation of Q. Elimination transfers by replacing each use of its opened clause with padding of the antecedent, use of the translated clause, and unpadding of the consequent. Transport transfers by Lemma I.6. Computation and assumptions are unchanged. This is an effective transformation of finite proofs.

The old calculus has binder congruence and the same padding equivalence, so structural induction gives its round trip. In the new calculus, uniform equivalence of two compound matrices lifts under D by DE and compound DI. At a guarded atomic matrix B, the remaining step is

$$\mathsf{D}_{x,y} B \dashv\vdash \mathsf{D}_{x,y} L(B).$$

The forward direction opens the source clause, unpads the target base, uses the clause, and pads the successor. For the reverse direction, the true guarded base proves $L(B(a, 0))$; the opened clause then gives $L(B(a, Sz))$ and hence $B(a, Sz)$. Atomic DI introduces $\mathsf{D}B$, ignoring its local assumption. These are all the cases in the new round-trip induction. An unused existential witness introduces or eliminates the old Q interpretation, using the witness 0. □

Theorem I.10 (Exact public consequence relation)**.** *For public Γ, A,*

$$\Gamma \vdash_{\mathcal{U}} A \iff \Gamma \vdash_{\mathcal{C}} A \iff \Gamma^{\circ} \vdash_{\mathrm{iPRA}} A^{\circ}.$$

Proof. Auxiliary soundness and public agreement give the rightward implication from the new calculus to iPRA. The pinned native–iPRA equivalence gives the old native derivation. Conversely, translate an old derivation by T and use both public round-trip equivalences, including the uniform equivalences for the retained assumptions, to recover the original public sequent. The second equivalence is the inherited native–iPRA bridge. □

Corollary I.11 (Bi-interpretability and equiconsistency)**.** *The ordinary realization of this public theory is bi-interpretable and equiconsistent with full first-order iPRA. Both numerical domains are the whole domain; equality is actual equality; and both numerical composites have the identity graph $x = y$.*

Proof. Compose T, J and their formula composites with the pinned bi-interpretation of the native calculus and iPRA. No term or numerical sort is changed by T or J, so the numerical composites remain the identity. The distinguished contradiction $0 = 1$ is unchanged and has the ordinary reading equivalent to falsity in iPRA. Exact public consequence therefore gives equiconsistency as well. □

I.7 Why the auxiliary distinction is essential

Proposition I.12 (The canonical-only composition would fail)**.** *If paired matrices throughout derivations were required to satisfy the public grammar, with* Q *resources only the corresponding opened clauses, the simultaneous restrictions would not even prove* $1 = 0$ *from* $0 = 1$*.*

Proof. Interpret terms in the standard natural numbers. At the root of a formula, declare every literal equation $0 = t$ true, and interpret every other equation by actual equality. Interpret public compounds by their ordinary conditional reading. Every atomic leaf under a public paired binder has an iterator on the left; consequently those leaves always receive actual equality, and their readings commute with substitution.

Computations, guarded atomic progress, compound case introduction, and the associated clause elimination are sound for this interpretation. Restricted transport is sound too. Its major $s = t$ has a syntactically nonzero left side, so it asserts actual equality. If an equation template instantiated at s has literal 0 on the left, that left template was constant 0: it could not have been the hole, since s is not literally 0. The target is then also true by convention. Otherwise transport preserves actual equality, unless its target has literal-zero left side and is true by convention anyway. Transport inside a public compound is ordinary extensional transport.

The assumption $0 = 1$ is true in this interpretation, while $1 = 0$ is false. This proves the stated nonderivability. The auxiliary induction used in Lemma I.4 is exactly the extra proof resource that invalidates this counterinterpretation. □

Remark I.13 (Exact checking contract)**.** The compact checker is a checker of auxiliary derivations with a *canonical public-goal contract*. Its soundness and completeness claim is Theorem I.10, for a goal in $\mathcal{G}$. A raw auxiliary goal need not be public, even when the program accepts its proof. The separately supplied strict checker includes and charges an explicit public-goal validator. A claim about the compact kernel alone must not silently include the validator, and a claim about the whole auxiliary syntax must not be substituted for the public bi-interpretation proved here.

J A uniform arity encoding and its checking theorem

The arithmetic and object logic remain the scalar-iteration Eq/D calculus specified above. The change is in its finite certificate representation. A paired formula is unary, an equation is binary, and an internal closure is ternary:

$$\mathsf{D}_{x,y}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s, t], \qquad \mathsf{Q}_y B \longleftrightarrow [B, B, B]. \tag{Arity encoding}$$

The three copies in a closure are representation redundancy: only the third has mathematical content. Their benefit is that tripling a unary major constructs exactly its closure. Tripling any other nonempty

formula array produces an array of arity at least six, which will denote truth. This makes the other major cases neutral without a separate test.

The previous generated-assumption and constructed-computation interfaces are inherited from the corpus [3, 4]. The new encoding makes a single arithmetic expression, $5 - \text{arity}$, perform the necessary depth adjustments. Its soundness depends on an explicit interpretation of surplus syntax. It would not be sound to drop arity checks while continuing to treat every surplus array as an unconstrained new connective.

J.1 The complete sorted interpretation of arrays

Write $\top$ for the fixed equation $0 = 0$. A protected numerical slot has width two. The numerical root has depth three: the coordinates $0, 1, 2$ all denote zero, and under p protected slots the depth is $3 + 2p$. Coordinates $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot, for $0 \leq j < p$. Formula depth is $2p$ under the same p slots. Negative coordinates admitted by the current global floor name rigid numerical parameters.

The decoder is sorted by the position in which a tree is used. Let ρ be its ordered environment of protected numerical slots. The notation (a, ρ) means prepend a slot of value a. A superscript $\sharp$ denotes a child decoded in the indicated environment.

Numerical arity	Meaning	Relevant child environment
0	0	No child
1: $[b]$	$Sb^\sharp$	$(0, 0, \rho)$
2	0	Annotations ignored
3: $[n, a, h]$	$I(n^\sharp, a^\sharp; v.h^\sharp)$	$(0, \rho)$ for n, a; (v, ρ) for h
≥ 4	0	All children ignored

Formula arity	Meaning	Relevant child environment
0	$\top$	No child
1: $[B]$	$\mathsf{D}_{x,y}B^\sharp$	(y, x, ρ)
2: $[s, t]$	$s^\sharp = t^\sharp$	ρ numerically
3: $[P, U, B]$	$\mathsf{Q}_y B^\sharp$	(y, ρ) for B
≥ 4	$\top$	All children ignored

The first two entries of a ternary formula are annotations. A canonical closure copies its body into both of them, as in (Arity encoding). Successfully admitted scalar formula padding also denotes $\top$. An internal closure has the ordinary meaning $B(0) \to \forall y B(y)$; it is not an equation and is not eligible for compound induction.

Canonical encoded object expressions use arrays and integer coordinates, respect these slots, and contain no high-arity node. The checker itself still generates high-arity truth entries in administrative contexts, even during a canonical proof. In particular, constructing a successor beneath positive bound coordinates requires passing through two dummy zero slots. Constructing an iterator requires one new slot: zero in its counter and seed, and its genuine state variable in the update. Rigid negative parameter names are unaffected by these shifts.

The additional raw shapes are certificate aliases with the displayed meanings. They are not independent predicate or function symbols added to the object theory. Our exact completeness statement concerns

the canonical native language. We do not require every possible raw spelling of a semantic alias to be interchangeable under the same raw certificate.

J.2 One depth calculation replaces the lookup table

For every array the opener recursively adds $5 - \text{arity}$ to its depth. The meaningful cases are

arity	1	2	3
depth increment	4	3	2.

Unary D adds two numerical binders; unary successor adds two dummy zero slots. Binary equality changes formula depth $2p$ to numerical depth $3 + 2p$. Ternary Q adds one numerical binder; ternary iteration adds one state slot, or one dummy slot in its counter and initializer. The same calculation therefore covers all relevant constructors.

Binary numerical annotations denote zero, so their parity-changing traversal has no mathematical effect. Empty arrays have no children. At arity at least four, the entire node already denotes a constant. Even a zero or negative depth increment below such a node cannot expose one of its ignored children as a meaningful variable. Traversal may still reject an alias; it cannot change its stipulated interpretation.

This is a semantic assertion, not raw idempotence of the opener. For example, at floor -1, opening a six-element array of zeros with image -1 can yield six copies of -1, while a later no-image opening of that result fails at a negative child depth. Both formula arrays denote $\top$. The proof below only concerns successful calls at meaningful sorted positions. Canonical input object expressions contain no surplus arrays; the administrative exports that generate them are justified separately as truth entries.

J.3 Opening and scope

Write $i(a, t, d)$ for the executable opener at floor k. It preserves a scalar when $k \leq a < d$. At a numerical position, every other relevant scalar is the distinguished substitution hole; the supplied image is checked at numerical root depth three before insertion. At a formula position scalar padding is truth and carries no numerical hole. Missing images at used numerical holes reject.

Lemma J.1 (Semantic opening)**.** *For every successful call at a meaningful formula root of depth $2p$ or numerical root of depth $3 + 2p$, with $p \geq 0$, decoding its result is capture-free filling of the distinguished hole in the decoded input template. These depths protect p numerical slots; admitted rigid parameters are preserved. If an image is semantically unused, zero may be chosen as its mathematical representative. Recursive calls at negative depths inside ignored annotations need no independent sorted interpretation.*

Proof. Induct on the finite raw tree. At a relevant scalar, the interval comparison is exactly the protected-coordinate test. At a meaningful unary, binary, or ternary node, the preceding tables supply precisely the required additional slots. At numerical binary annotations, high-arity nodes, and ignored closure fields, the relevant mathematical meaning is constant or absent, so the conclusion is immediate regardless of traversal inside the annotation.

An inserted image is checked at its own numerical root. Its free parameters are rigid negative names, and it has no unbound positive coordinate there. Insertion beneath protected slots therefore captures none of its parameters; its internal binders retain their relative indices. This proves the inductive step for substitution as well as for mere traversal. □

A successful no-image check consequently preserves the interpretation of its checked closed expression. Raw structural equality of checked outputs implies equality of their decoded meanings. The checker does not infer equality merely from a semantic comparison of two different programs or numerical terms.

The global floor decreases by one at each non-hypothesis continuation. If its old value is K, the integer $K-1$ is the designated fresh eigenparameter. With integer coordinates it is the only new name. If fractional coordinates are accepted, other names in $[K-1, K)$ may also occur in the finite certificate. These are additional fresh parameters. They must not be silently equated to the designated integer eigenparameter.

Previously checked relevant goals and retained hypotheses mention none of these new names: their earlier no-image checks would reject a relevant numerical occurrence below the old floor. Coordinates inside ignored annotations do not contribute free variables to the decoded formula. This is the freshness invariant used in every rule below.

J.4 A computation hypothesis only needs a zero instance

The shared continuation can open a generated arithmetic equation at a fresh parameter. That equation need not be a theorem uniformly in the new parameter. The following admissible rule is sufficient.

Lemma J.2 (Supply with a known zero instance)**.** *Suppose $\vec{a}$ is fresh for Γ, F, the calculus proves $H(\vec{0})$, and $\Gamma, H(\vec{a}) \vdash F$. Then $\Gamma \vdash F$.*

Proof. Substitute zero for every member of $\vec{a}$ in the second derivation. Freshness leaves Γ, F unchanged. Cut in the derivation of $H(\vec{0})$. Both substitution and cut are admissible in the native calculus. This is a finite proof transformation and does not use a consistency assumption. □

All names newly admitted when the floor changes belong to $\vec{a}$; for noninteger aliases this includes the finite collection in $[K-1, K)$ that actually occurs. On decoding, their simultaneous zero substitution agrees with opening the generated expression at zero using the old floor. Previously admitted parameters are preserved. New rigid names and unprotected placeholders both become zero. The opener may keep different raw zero spellings, but their decoded values agree.

The constructed-computation packet supplies a triple $s = [n, b, r]$ and a mode u. In successor mode its generated equation has raw form

$$\big[\,[[n], b, r],\ \ i(r, s, 3)\,\big], \tag{Step supply}$$

while in zero mode it has form

$$\big[\,[u, b, r],\ \ b\,\big]. \tag{Zero supply}$$

Here successor mode means that u is truthy; the zero mode has falsy u. These expressions describe actual arrays, before their final opening. In (Step supply) the counter is constructed as a successor, rather than accepted on an unsupported assertion that it is one. This construction is inherited from the corresponding draft interface [4].

Lemma J.3 (Sound constructed computation)**.** *Every successful arithmetic supply has a decoded zero instance derivable from the two scalar-iteration computation schemas.*

Proof. First apply the simultaneous zero substitution described above. In zero mode every successful falsy numerical counter denotes zero, so (Zero supply) is an $I0$ instance. In the exact-list profile the relevant accepted aliases are an empty array, numeric zero, false, and numerically equal zero values.

Null, strings, and dictionaries cannot enter a meaningful numerical position through an unchecked coercion.

In successor mode the left counter is an actual successor. If the update uses its state in a semantically relevant position, opening it invokes the no-image check of the recursive source s at numerical root three. This checks the entire source, including its update. Only coordinates three and four are aliases for its single state slot. A genuinely escaping coordinate in that update is revisited during the recursive-image check and rejects. Its counter and initializer are likewise closed with their prescribed dummy zero slot.

Inserting the extra dummy slots of the displayed successor does not alter such a checked predecessor: extra unbound slots have value zero, and actual free parameters are rigid. Lemma J.1 therefore identifies the right side of (Step supply) with the update applied to exactly the recursive predecessor value occurring on its left. This is the IS schema.

If the update has no semantically relevant state occurrence, its decoded value is constant in that variable. Its zero-normalized value is the same in the source update and on the right, independently of the unused recursive image; this is again an IS instance. Traversal inside an ignored annotation may still inspect the image and reject. That removes a certificate and does not invalidate the argument for a successful one.

Finally, successful three-field source extraction cannot hide another JSON operator. A scalar is not iterable. A three-character string, or a three-key JSON dictionary, necessarily puts a nonempty string in a relevant source field, which the inspected opener rejects. The successful source is therefore the stated array triple. □

The recursive-image check in the state-dependent case is essential. Merely opening a final displayed equation at zero would not establish that its two state interpretations agree. Combining Lemma J.3 with Lemma J.2 justifies both actual image choices of the common continuation, including the branch that retains a fresh-parameter hypothesis.

J.5 Audit of the logical continuations

Hypothesis closure. The initial opener check and membership comparison select the same decoded context formula. Additional aliases can make a spelling less convenient, but cannot identify an equation with a false denotation solely by semantic evaluation.

Equality transport. The major must match $[s, u]$, and opening the proposed formula template at u must match the target. The continuation checks its opening at s. Lemma J.1 gives ordinary transport along the decoded equality. The argument applies to compound formulas, internal clauses, and truth aliases as well as equations.

Closure specialization. The closure pattern is the triplicate $[t, t, t]$. The guard forces the old instance to have numerical image zero; the new instance may have an arbitrary numerical image. This is exactly $B(0) \to \forall y B(y)$. The zero guard is indispensable. Without it, a closure with false zero instance and true successor instances would turn a proved successor instance into falsity.

Paired elimination. The export is always three concatenated copies of the major. A unary major $[B]$ therefore exports $[B, B, B]$. All three copies are opened identically; the third copy has precisely the

witness instance required by paired elimination. The newly allocated witness is fresh for the retained context and conclusion.

An equation major instead exports arity six; a closure major exports arity nine; a larger array exports still larger arity. These are all truth aliases. Thus failed elimination endpoint claims have only the effect of adjoining a theorem. A numeric scalar major, if accepted as formula padding, exports scalar padding after multiplication and also denotes truth. Unsupported types reject.

A ternary formula with unequal annotations still denotes the closure of its third child. It fails the triplicate specialization pattern unless the three fields match. This loses a possible raw spelling, not a mathematical soundness condition. Canonical compilation uses triplicate closures throughout.

Paired introduction. The call `i(*a,t,2)` uses the function's argument count to enforce the relevant outer arity. For a unary goal $a = [B]$, it is exactly `i(B,t,2)`: the universal slot is protected while the existential witness is opened. For arity at least two, there are too many arguments and the call rejects. An empty goal can instead make the legal two-argument call `i(t,2)`. Its original target was already $\top$, so accepting it cannot add a false theorem.

For the unary goal, the subsequent successor opening gives the intended premise $B(t, Sz)$. The local hypothesis is $B(t, 0)$ in case mode and $B(t, z)$ in progress mode. The freshness invariant supplies the usual eigenvariable conditions.

Induction remains restricted. Among meaningful nonconstant matrix constructors, only binary equality has even arity. Unary paired formulas and ternary closures have odd arity and receive case introduction. Empty or surplus arrays that reach the progress branch denote the fixed equation $0 = 0$, whose induction is valid. Scalar padding that cannot provide a length rejects. The broader certificate alphabet therefore adds no induction on a quantified matrix.

Additional fresh parameters. An accepted subderivation can contain newly admitted fractional names, but the older context and target do not depend on them. Replace these auxiliary names by zero when closing the rule, retaining its designated integer eigenvariable where that rule needs one. The common opening of an already checked retained goal leaves its decoded meaning unchanged. This handles the actual global-floor convention without pretending that every admissible negative number is an integer.

J.6 Canonical completeness, including zero witnesses

Rename a finite native proof's eigenvariables to fresh negative integers. Encode its terms with the slot conventions of Section J.1; encode paired formulas as unary and closures as triplicate. Hypothesis leaves consume no instruction. Transport, closure specialization, and paired elimination have their displayed packets. Arithmetic computation leaves use (Zero supply) or (Step supply) and then close the target by hypothesis lookup.

One representational issue requires an actual proof transformation. The packet's truthy witness field selects introduction. A literal zero witness therefore cannot simply be inserted into that field. Define

$$J(t) = I(0, t; v.0).$$

The computation axiom proves $J(t) = t$, and $J(t)$ has a nonempty array representation even when its value is zero. To introduce with a falsy witness spelling t, introduce instead with $J(t)$: transport its

retained assumption to the assumption using t, use the original premise derivation, and transport the resulting successor instance back to the instance using $J(t)$. Derived symmetry supplies the reverse transport. This is a finite transformation of the premise, and the paired conclusion itself is unchanged. If the existential coordinate is unused, any closed truthy witness works without a conversion.

The scalar coordinate 1 is also a semantic zero at numerical root three, but that observation alone is not a completeness proof. Closure specialization and case introduction insert literal raw zero, and distinct zero aliases need not be interchangeable under a fixed raw proof spelling. The computation-backed box gives the required derivation instead of assuming representational interchangeability.

The mode zero selects progress at an equation. A different falsy mode, such as the empty array, selects case. These are certificate control tags, not numerical assertions about the witness. All steps in the compilation are finite and effective.

Theorem J.4 (Conservativity and completeness of the arity checker)**.** *Every accepted certificate has its decoded conclusion provable in the ordinary realization of the native calculus. For every canonical native end sequent, the internal checking judgment and the native calculus have the same consequence relation, in the idealized unbounded integer-and-array implementation. In particular, the public empty-context driver recognizes exactly the canonical closed theorems. The translation in each direction is effective on finite derivations.*

Proof. For soundness, induct on the finite accepting call tree. Hypothesis closure uses Lemma J.1; the logical cases are the audited continuations above. Arithmetic supply uses Lemmas J.3 and J.2. Every discarded annotation is accounted for by its zero or truth interpretation, and every fresh parameter by the scope invariant. For completeness, compile a finite native derivation by the preceding construction, including the boxed-witness transformation. Its canonical input expressions require no surplus-arity aliases or fractional names; generated administrative surplus arrays have exactly the truth meaning already proved. This proves both implications without using a finite test suite as a substitute for induction on derivations. □

The unchanged native calculus is bi-interpretable with full first-order iPRA by the inherited arithmetic and logical bridge [3, 2]. In particular,

$$(\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

The whole numerical domain and actual equality are retained. Its internally proved numerical round trips and its formula-composite derivations therefore transfer through Theorem J.4. We have changed the checked representation, not replaced bi-interpretability by standard-model agreement or supplied a new proof of the scalar predecessor theorem.

J.7 JSON and finite-runtime qualifications

The primary profile uses an ordinary list-type test. The smaller byte profile instead tests multiplication by zero against an empty list. Among standard JSON runtime values the latter recognizes exactly lists: numeric values and booleans produce numeric zero, strings produce an empty string, and null and dictionaries raise rather than becoming arrays. This argument assumes ordinary decoded JSON values, not arbitrary Python objects with overloaded methods.

Finite nonnegative real coordinates have the same interval interpretation: $[0,3)$ is zero, and $[3{+}2j, 5{+}2j)$ names slot j. Finite negative coordinates in the admitted range are additional rigid names. Equal Python numerical values have the same decoded role, including Boolean aliases and signed zero.

Nonfinite values admitted by Python's permissive JSON reader cannot survive a relevant closed numerical check. They may be filled as template holes or ignored inside a constant annotation; neither makes them unchecked parameters.

The alternative opener based on generic iteration has a separate runtime qualification. Empty strings and empty dictionaries traverse as empty arrays; a dictionary whose sole key is empty traverses as a unary array with an empty child. Dictionary values are not visited. Nonempty strings can rely on CPython's recursion limit to reject, as also occurs in a current corpus comparison profile [11]. The idealized structural-termination claim for the exact-list profile must not be transferred to that generic profile unchanged.

All executable profiles have ordinary finite memory, stack, and input limits. These can reject a sufficiently large otherwise valid proof. The formal consequence relation is the unbounded finite-certificate calculus; the delivered program checks certificates within its runtime resources. The measurements charge the entire delivered checker, while the semantic proof and canonical producer are supplied separately.

K Complete new checker listings

Each listing is a complete acceptance module. The executable files in `code/` are authoritative for the measurements and can be run independently. The expanded explanatory implementation is additional documentation, outside every measured trusted module.

K.1 Inferred-major token profile

```
from json import load
k = 0

def i(a, t=0j, d=0):
    try:
        return [i(b, t, d + 5 - len(a)) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    while i(a) not in g:
        m, s, u, t = q()
        if i(t, u) == a:
            m = [s, u] if s else [t] * 3
            a = i(t, s)
        h = c(m, g) or m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a,
     t, 2))
        k -= 1
        g += (i(h, (m or len(a) == 2) and k),)
        a = i(a, [k])
q = load(open(0)).pop
c(q(), ())
print('ACCEPT')
```

K.2 Inferred-major AST profile

```
from json import load

def c(a, g=(0,), k=0):

    def i(a, t=None, d=0):
        try:
            return [i(b, t, d + 5 - len(a)) for b in a]
        except:
            return a if k <= a < d else d % 2 and i(t, d=3)
    while i(a) not in g:
        m, s, u, t = q()
        if i(t, u) == a:
            m = [s, u] if s else [t] * 3
            a = i(t, s)
        h = c(m, g, k) or m * 3 or ([[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t,
     2)))
        k -= 1
        g += (i(h, (m or len(a) == 2) and k),)
        a = i(a, [k])
c((q := load(open(0)).pop)())
print('ACCEPT')
```

K.3 Truthy-result byte profile

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d else d % 2
     and i(t, d=3)
    return i(a) in g or [((h := (c(m, g, k) and m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else
      u]) or (a := i(*a, t, 2)))), c(i((m, a) == (s and [s, u] or [t] * 3, i(t, u)) and i(t, s) or a, [(k := (~-k
     ))]), g + [i(h, (m or ~len(a) % 2) and k)], k)) for m, s, u, t in [q()]]
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

K.4 Byte profile, ordinary decrement

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d else d % 2
     and i(t, d=3)
    return i(a) in g or [((h := (c(m, g, k) and m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else
      u]) or (a := i(*a, t, 2)))), c(i((m, a) == (s and [s, u] or [t] * 3, i(t, u)) and i(t, s) or a, [(k := (k -
      1))]), g + [i(h, (m or ~len(a) % 2) and k)], k)) for m, s, u, t in [q()]]
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

K.5 Native public-goal profile

```
import json

def c(a, g=[0], k=0, v=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] and v <= (len(a) in (1, 2 + d %
     2)) else a if k <= a < d and v <= (type(a) is int and d % 2 and (not a or a > 1 == a % 2)) else d % 2 | v
     and i(t, d=3)
    if i(a) in g:
        return
    v = 0
    m, s, u, t = q()
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    c(i(m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a, [(k := (k - 1))]), g + [i(h, (m or
     len(a) % 2 == u) and k)], k)
q = json.load(open(0)).pop
c(q(), v=1)
print('ACCEPT')
```

References

[1] ProveIt contributors. Concise presentations of intuitionistic primitive recursive arithmetic. Pinned snapshot, 12 November 2026. Pinned primary source.

[2] ProveIt contributors. Collected research packages and current formalization. Pinned primary source.

[3] ProveIt contributors. Inherited arity encoding and generated-assumption proof; reproduced in this report. Pinned primary source.

[4] ProveIt contributors. Constructed-computation certificate interface. Pinned primary source.

[5] ProveIt contributors. Guarded core and deterministic introduction. Pinned primary source.

[6] ProveIt contributors. Literal-zero-source admissibility and source dispatch. Pinned primary source.

[7] ProveIt contributors. Lean formalization and source-admissibility results, as recorded in the pinned README. Pinned primary source.

[8] ProveIt contributors. Inherited scalar predecessor bootstrap; exact attributed appendix reproduced in this report. Pinned primary source.

[9] ProveIt contributors. Inherited equality reflection, pairing, and primitive-recursion compilation appendices; reproduced in this report. Pinned primary source.

[10] ProveIt contributors. Inherited native arithmetic and proof-certificate foundations. Pinned primary source.

[11] ProveIt contributors. Transient-seed arithmetic development. Pinned primary source.